МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт искусственного интеллекта
Кафедра проблем управления

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине «Программное обеспечение мехатронных
и робототехнических систем»

Тема лабораторной работы: «Отладка программного обеспечения робототехнических систем с использованием виртуального моделирования»

Студент группы: КРБО–01–23 Абакумов А. Я.
Балышев Е.Ю.
Егоров М.Ю.
Сгибнев Д.Р.
Казаков А.И.

Преподаватель: ст. преподаватель Морозов А. А.

Работа представлена к защите: «_____» ____2026 г.

Москва 2026

ЦЕЛЬ РАБОТЫ

Получение навыков моделирования объекта управления в промышленных системах автоматического управления, создание повторно используемых функциональных блоков и исследование переходных процессов объекта с ПИ–регулятором и без регулятора.

ЗАДАНИЕ

Создать виртуальную систему управления двигателем постоянного тока, включающую модель объекта управления, ПИ–регулятор, сумматор и отрицательную обратную связь. Модель должна быть реализована в среде Automation Studio на языке ANSI C с применением функциональных блоков FB_Integrator, FB_Motor и FB_Regulator.

Для экспериментального исследования сформировать ступенчатое задающее воздействие, подключить второй экземпляр двигателя без регулятора и получить средством Trace графики уставки и скоростей двух двигателей при различных настройках коэффициентов ПИ–регулятора.

ОСНОВНАЯ ЧАСТЬ

4.1 Описание действий и использованного аппарата

Работа была выполнена в среде B&R Automation Studio 4.12 с использованием виртуального контроллера ARsim. Программное обеспечение системы управления было разработано на языке ANSI C. Для наблюдения за переменными и регистрации переходных процессов применялись средства Watch и Trace.

В проект была добавлена пользовательская библиотека, содержащая функциональные блоки интегратора, модели двигателя постоянного тока и ПИ–регулятора. В основной программе были созданы два экземпляра модели двигателя: fb_motor работал в замкнутом контуре с р егулятором, а fb_motor_2 использовался для сравнения переходного процесса без регулятора.

В приведённом листинге основная программа рассчитана на циклический класс Cyclic #3 с периодом 50 мс. Внутренний шаг модели составляет 2 мс; цикл из 25 расчётов за одно выполнение программы обеспечивает соотношение 25 · 0,002 = 0,05 с. Экспериментальные осциллограммы далее используются для сравнения характера переходных процессов; соответствие их временного масштаба приведённому листингу отдельно не подтверждено.

При выполнении работы были использованы:

– среда разработки B&R Automation Studio 4.12;

– виртуальная целевая система ARsim;

– язык программирования ANSI C;

– циклический класс Cyclic #3 с периодом 50 мс;

– средство Trace для регистрации переменных Speed, fb_motor.w и fb_motor_2.w.

Исходные тексты интерфейсов функциональных блоков, переменных и программных модулей приведены в приложениях А–Ж.

4.2 Схема модели объекта управления и регулятора

Двигатель представлен апериодическим звеном первого порядка, передаточная функция которого приведена в выражении (1):

$$W(s) = \frac{1}{k_e (T_м s + 1)} \tag{1}$$

Соответствующее дифференциальное уравнение имеет вид (2):

$$\frac{dw}{dt} = \frac{u/k_e - w}{T_м} \tag{2}$$

где u –напряжение, w –частота вращения, ke –постоянная ЭДС, Тм –электромеханическая постоянная времени.

При численном интегрировании методом Эйлера получено разностное уравнение (3), которое и реализовано в программном коде функционального блока FB_Motor:

$$w_{k+1} = w_k + \frac{dt}{T_м}\left(\frac{u_k}{k_e} - w_k\right) \quad (3)$$

Ошибка регулирования определяется выражением (4):

$$e = Speed - w \quad (4)$$

ПИ–регулятор формирует управляющее воздействие из пропорциональной и интегральной составляющих. В модели предусмотрены ограничения напряжения и коррекция интегральной составляющей при насыщении.

Для двигателя без регулятора в рассматриваемом варианте используется нормирование входного напряжения (5):

$$u_2 = Speed \cdot k_e \quad (5)$$

Оно обеспечивает установившуюся скорость w2 = Speed для данной модели. Такое нормирование следует явно указать, поскольку непосредственная подача числа 60 на вход напряжения не эквивалентна заданию скорости 60 об/мин.

Замкнутая система управления была построена по принципу отрицательной обратной связи (рисунок 1). Из уставки скорости Speed вычиталась фактическая скорость fb_motor.w. Полученное рассогласование поступало на ПИ–регулятор, выход которого задавал входное напряжение модели двигателя.

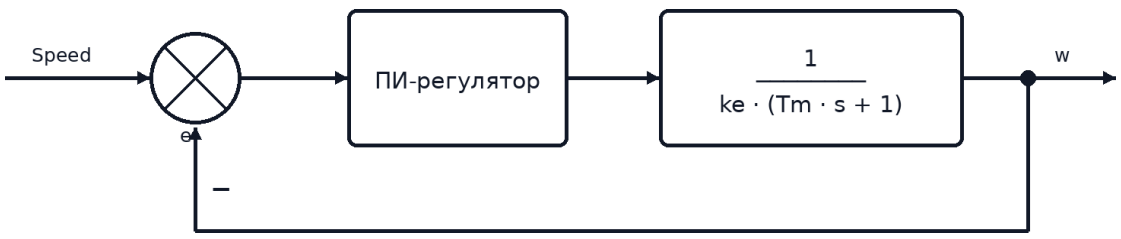


Рисунок 1. Структура замкнутой системы управления

В программной модели апериодическое звено первого порядка было реализовано с помощью интегратора и обратной связи по скорости (рисунок 2). Дополнительно вычислялось угловое положение вала phi.

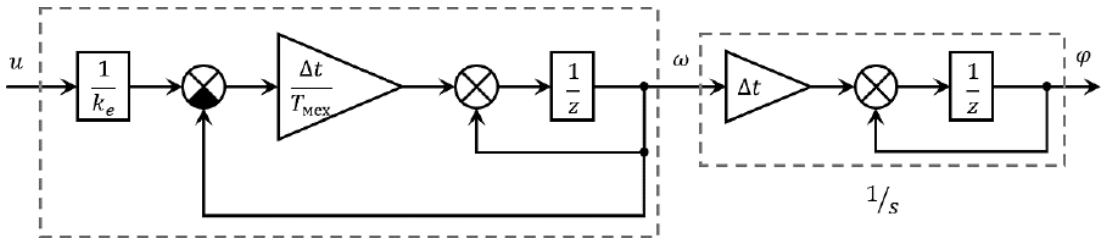


Рисунок 2. Структура модели двигателя постоянного тока

ПИ–регулятор содержал пропорциональную и интегральную составляющие (рисунок 3). После пропорционального звена и на общем выходе применялись ограничения по абсолютному значению напряжения. Для предотвращения накопления интегральной составляющей при насыщении была реализована обратная коррекция iyOld.

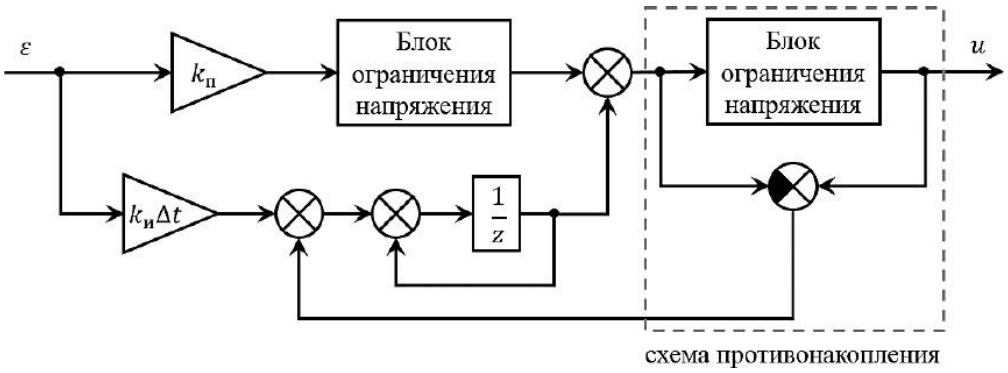


Рисунок 3. Структура ПИ–регулятора со схемой противонакопления

Структурная схема интегрирующего звена, общего для моделей двигателя и регулятора, приведена на рисунке 4.

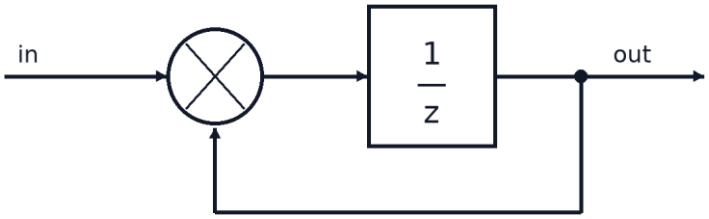


Рисунок 4. Структурная схема интегратора

Расчёт коэффициентов регулятора

Расчёт коэффициентов ПИ–регулятора был выполнен методом обратной задачи динамики. Желаемая передаточная функция замкнутого контура принималась в виде апериодического звена с постоянной времени Тж, что даёт передаточную функцию регулятора (6):

$$W_{\text{р}}(s) = \frac{1}{T_{\text{ж}}\, s\, W_{\text{о}}(s)} = \frac{k_e\, T_{\text{м}}}{T_{\text{ж}}} + \frac{k_e}{T_{\text{ж}}} \cdot \frac{1}{s} \tag{6}$$

Отсюда коэффициенты регулятора определяются выражениями (7):

$$k_{\text{п}} = \frac{k_e\, T_{\text{м}}}{T_{\text{ж}}}, \qquad k_{\text{и}} = \frac{k_e}{T_{\text{ж}}} \tag{7}$$

При параметрах объекта ke = 0,00096 и Тм = 0,04 с и принятой желаемой постоянной времени Тж = 0,006 с получены значения (8):

$$k_{\text{п}} = \frac{0{,}00096 \cdot 0{,}04}{0{,}006} = 0{,}0064, \qquad k_{\text{и}} = \frac{0{,}00096}{0{,}006} = 0{,}16 \tag{8}$$

Полученные коэффициенты kп = 0,0064 и kи = 0,16 заданы в примере инициализации (приложение В). Дополнительно исследованы настройки для Тж = 1 с, принятой в методичке, и Тж = 0,2 с. По формулам (7) получены соответственно пары kп = 0,0000384, kи = 0,00096 и kп = 0,000192, kи = 0,0048. В каждом опыте изменялись оба коэффициента; параметры двигателей сохранялись.

4.3 Структурная схема конечного автомата

Формирование испытательного воздействия было представлено как конечный автомат, состояние которого определялось переменными Enable и counter (рисунок 5). При Enable = 0 счётчик и уставка удерживались в нуле. После включения системы автомат последовательно проходил три временных состояния.

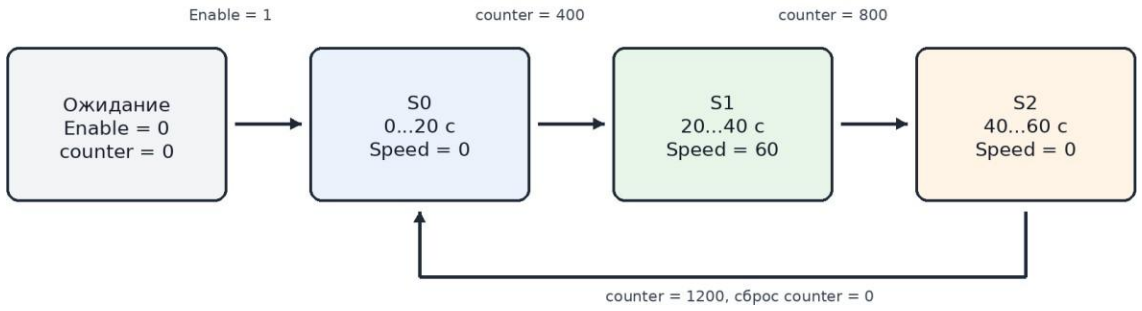


Рисунок 5. Конечный автомат формирования ступенчатого воздействия

4.4 Описание состояний

Состояния конечного автомата и соответствующие им значения уставки приведены в таблице 1.

Таблица 1 –Состояния конечного автомата формирования уставки

Состояние	Условие	Уставка Speed	Назначение
Ожидание	Enable = 0	0 об/мин	Система отключена, counter сбрасывается
S0	0 ≤ counter < 400	0 об/мин	Начальный участок длительностью 20 с
S1	400 ≤ counter < 800	60 об/мин	Ступенчатое воздействие длительностью 20 с
S2	800 ≤ counter < 1200	0 об/мин	Наблюдение процесса остановки в течение 20 с

После достижения counter = 1200 счётчик сбрасывался, и последовательность повторялась. Поскольку период Cyclic #3 равен 50 мс, значения 400, 800 и 1200 соответствуют моментам времени 20, 40 и 60 с.

4.5 Описание функциональных блоков

Функциональный блок FB_Integrator выполнял численное интегрирование методом прямоугольников. На каждом расчётном шаге выходное значение увеличивалось на произведение входа и шага времени: out(k) = out(k−1) + in(k) · dt. Поля блока приведены в таблице 2.

Таблица 2 –Параметры функционального блока FB_Integrator

Поле	Тип	Назначение
in	REAL	Вход интегрирующего звена
out	REAL	Накопленное выходное значение
dt	REAL	Шаг расчёта, равный 0,002 с

Функциональный блок FB_Motor моделировал динамику двигателя постоянного тока как апериодическое звено первого порядка. По входному напряжению u вычислялась установившаяся скорость u/ke, после чего интегратор формировал текущую скорость w с постоянной времени Тм. Угловое положение phi определялось интегрированием скорости, предварительно переведённой из оборотов в минуту в радианы в секунду. Поля блока приведены в таблице 3.

Таблица 3 –Параметры функционального блока FB_Motor

Поле	Тип	Назначение
u	REAL	Входное напряжение двигателя, В
w	REAL	Частота вращения, об/мин
phi	REAL	Угловое положение вала, рад
integrator	FB_Integrator	Интегратор модели скорости
Tm	REAL	Электромеханическая постоянная времени
ke	REAL	Постоянная ЭДС двигателя
dt	REAL	Шаг расчёта модели

Функциональный блок FB_Regulator реализовывал ПИ–закон управления. Пропорциональная составляющая рассчитывалась как e · k_p, а интегральная –путём интегрирования e · k_i. Ограничители не позволяли выходному напряжению превысить ±24 В. Разность между ограниченным и неограниченным выходами использовалась в цепи противонакопления интегратора. Поля блока приведены в таблице 4.

Таблица 4 –Параметры функционального блока FB_Regulator

Поле	Тип	Назначение
e	REAL	Ошибка регулирования Speed − w
Поле	**Тип**	**Назначение**
u	REAL	Выходное управляющее напряжение
k_p	REAL	Пропорциональный коэффициент
k_i	REAL	Интегральный коэффициент
integrator	FB_Integrator	Интегратор И–составляющей
iyOld	REAL	Коррекция схемы противонакопления
max_abs_value	REAL	Абсолютная граница напряжения
dt	REAL	Шаг расчёта регулятора

Численные значения параметров моделей и коэффициенты исходной настройки регулятора приведены в таблице 5. Для дополнительных опытов использованы пары коэффициентов, указанные в разделе 4.7.

Таблица 5 –Параметры моделей объекта и регулятора

Параметр	Значение	Назначение
dt	0,002 с	Внутренний шаг моделирования
Тм	0,04 с	Постоянная времени двигателя
ke	0,00096	Постоянная ЭДС двигателя
k_p	0,0064	Пропорциональный коэффициент ПИ–регулятора
k_i	0,16	Интегральный коэффициент ПИ–регулятора
max_abs_value	24 В	Ограничение управляющего напряжения

4.6 Описание структуры программного обеспечения

Программное обеспечение состояло из основной программы Program и пользовательской библиотеки MotorControl. В Init выполнялась инициализация параметров и начальных состояний. В Cyclic формировалась уставка, вычислялась ошибка регулирования и последовательно вызывались функциональные блоки регулятора и двух двигателей. Структура программного обеспечения приведена на рисунке 6.

Для двигателя с регулятором вход формировался выходом fb_controller.u. Для второго двигателя использовался нормированный вход Speed · ke, что обеспечило совпадение установившегося значения скорости с уставкой при заданной постоянной ke. Оба двигателя рассчитывались с одинаковыми параметрами, поэтому различие графиков определялось способом формирования управляющего воздействия.

4.7 Результаты экспериментальных исследований

В конфигурацию Trace включены переменные Program:Speed, Program:fb_motor.w и Program:fb_motor_2.w. В момент около 20 с уставка изменяется с 0 до 60 об/мин, а около 40 с возвращается к нулю. На рисунках 7–9 верхняя красная кривая показывает скорость двигателя с регулятором, средняя зелёная –без регулятора, нижняя синяя –уставку. Исходная осциллограмма сохранена на рисунке 7; дополнительные опыты представлены на рисунках 8 и 9.

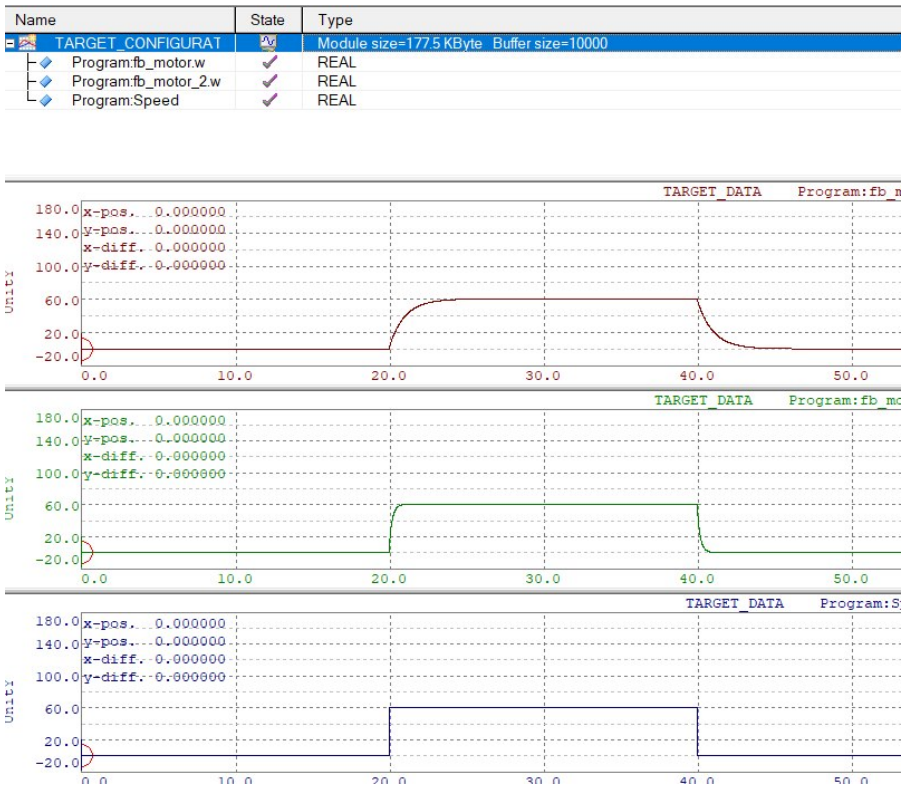


Рисунок 6. Переходные процессы при kп = 0,0064 и kи = 0,16 (исходная настройка)

При исходной настройке (рисунок 7) регулируемый двигатель быстро достигает уровня задания 60 об/мин и удерживает его до снятия уставки. После 40 с скорость быстро возвращается к нулю. На представленном масштабе времени заметного перерегулирования не наблюдается.

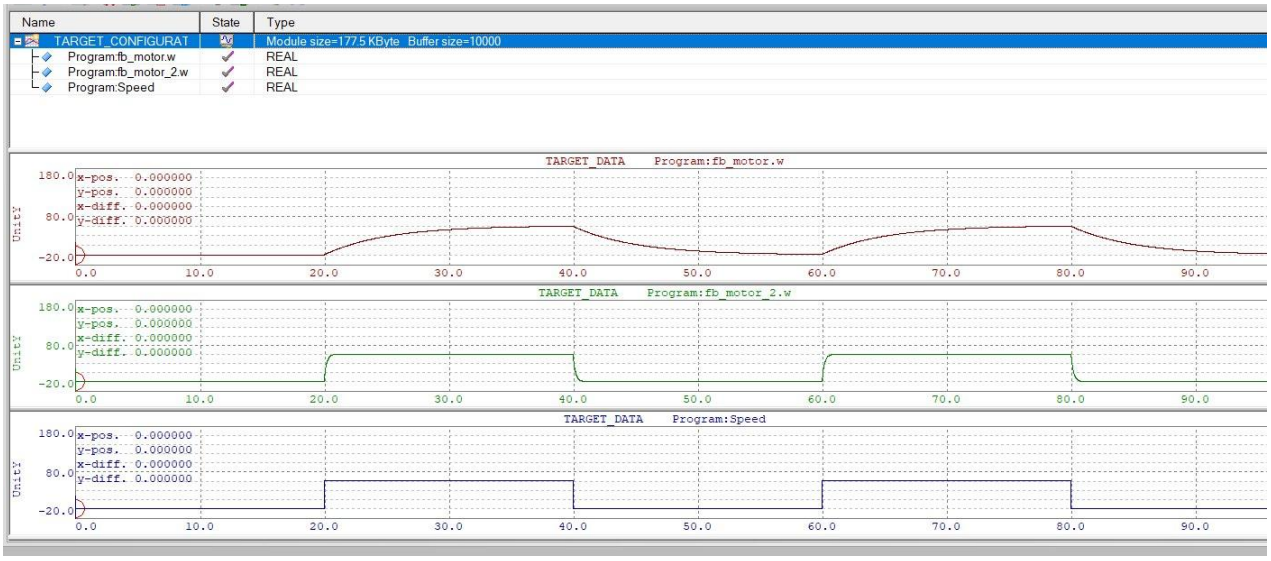


Рисунок 7. Переходные процессы при kп = 0,0000384 и kи = 0,00096

При наименьших коэффициентах (рисунок 8) скорость регулируемого двигателя нарастает медленно. По показанию курсора в момент 39 с она составляет около 32,03 об/мин при уставке 60 об/мин. Следовательно, за время действия задания двигатель не успевает достигнуть требуемой скорости.После снятия уставки наблюдается медленное снижение скорости. Ошибка в момент 39 с составляет около 27,97 об/мин; она характеризует незавершённый переходный процесс, а не установившуюся ошибку.

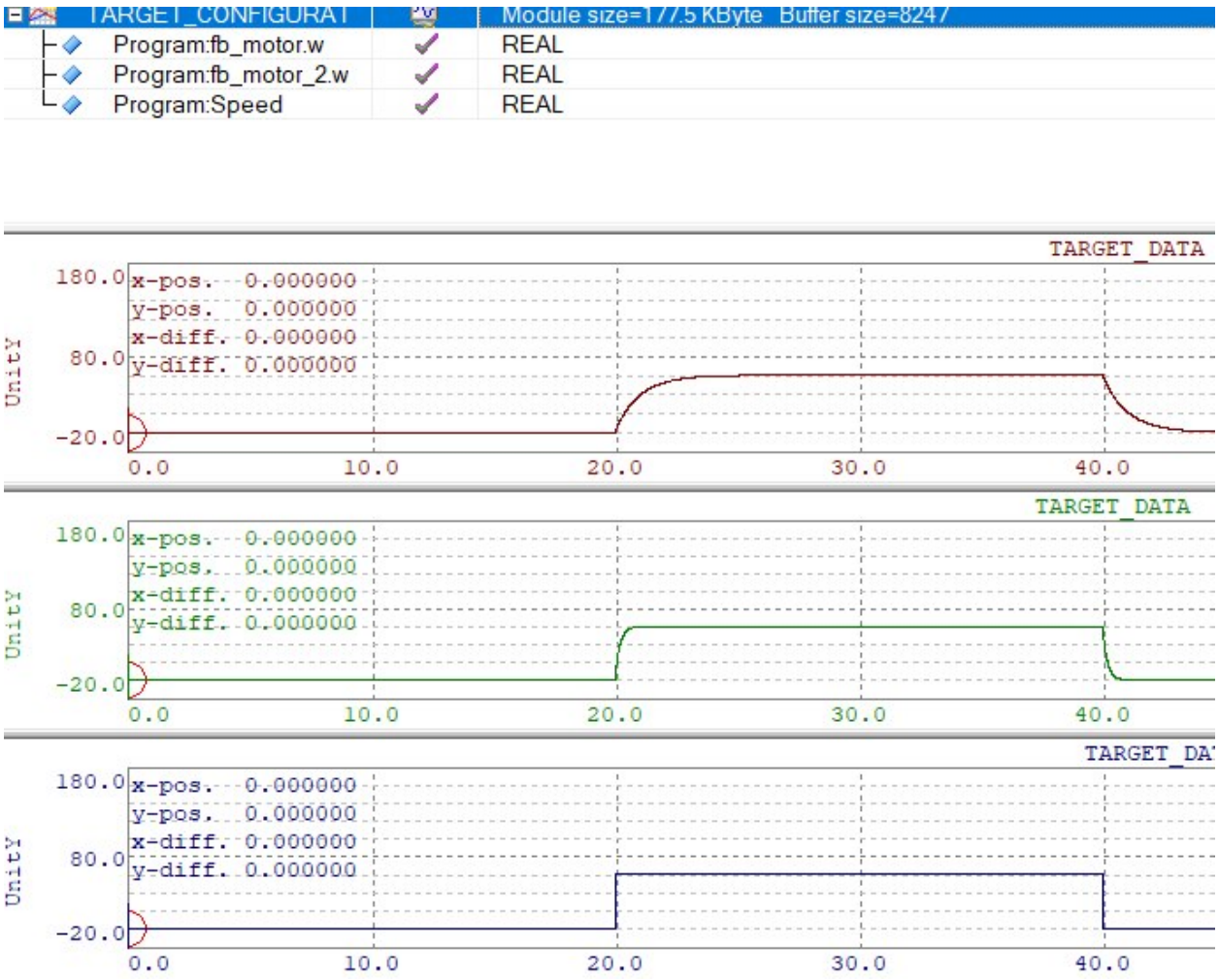


Рисунок 8. Переходные процессы при kп = 0,000192 и kи = 0,0048

При увеличении обоих коэффициентов в пять раз относительно рисунка 8 регулируемый двигатель разгоняется заметно быстрее и к концу интервала действия задания приближается к 60 об/мин (рисунок 9). После 40 с скорость плавно снижается к нулю. Заметного превышения уставки на графике не наблюдается. Точное время установления по данному масштабу изображения не определялось.

Сравнение трёх настроек показывает, что совместное увеличение kп и kи при сохранении отношения kп/kи = 0,04 с повышает быстродействие в исследованном диапазоне. Наиболее медленный процесс получен при kп = 0,0000384 и kи = 0,00096, промежуточный –при kп = 0,000192 и kи = 0,0048, наиболее быстрый –при исходных коэффициентах kп = 0,0064 и kи = 0,16.

Кривая двигателя без регулятора во всех опытах сохраняет одинаковый характер: скорость выходит на уровень около 60 об/мин и снижается после снятия задания. Этот результат связан с нормированием входа u2 = Speed · ke. Поэтому в данном исследовании сравнивается прежде всего динамика разгона и остановки, а не преимущество по установившейся ошибке.

Временные отметки на осциллограммах относятся к регистрации Trace. Их нельзя непосредственно отождествлять с расчётной постоянной времени Тж: наблюдаемые длительности отличаются от ожидаемых для листинга с 25 внутренними расчётами за цикл. По представленным данным сделан качественный выводо влиянии коэффициентов; точно соответствие модельного времени времени регистрации не устанавливалось.

ВЫВОДЫ

В ходе лабораторной работы была разработана виртуальная система управления двигателем постоянного тока. Были созданы функциональные блоки интегратора, модели двигателя и ПИ–регулятора с ограничением выходного напряжения и схемой противонакопления. Коэффициенты регулятора были рассчитаны методом обратной задачи динамики.

В основной программе объединены замкнутый контур регулирования и второй двигатель без регулятора. С помощью счётчика сформировано ступенчатое воздействие 0–60–0 об/мин с переключениями около 20 и 40 с. Средством Trace получены графики для трёх пар коэффициентов ПИ–регулятора.

Сравнение осциллограмм показало, что при совместном увеличении коэффициентов в выбранном соотношении переходный процесс ускоряется. При наименьшей паре коэффициентов скорость не достигает уставки за время её действия; при промежуточной настройке приближается к 60 об/мин, а при исходной –быстро достигает заданного уровня. Заметного перерегулирования на представленных графиках не наблюдается. Динамика двигателя без регулятора при изменении коэффициентов ПИ–регулятора сохраняется. Применение функциональных блоков обеспечило модульную и повторно используемую структуру программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Программное обеспечение мехатронных и робототехнических систем : методические указания к лабораторному практикуму. –М. : РТУ МИРЭА, 2021. –60 с. –Текст : непосредственный.

2. Медведев, М. Ю. Программирование промышленных контроллеров / М. Ю. Медведев, В. Х. Пшихопов. –СПб. : Лань, 2011. –288 с. –Текст : непосредственный.

3. Лукинов, А. П. Проектирование мехатронных и робототехнических устройств / А. П. Лукинов. –СПб. : Лань, 2012. –608 с. –Текст : непосредственный.

4. Федоров, Ю. Н. Справочник инженера по АСУТП: проектирование и разработка / Ю. Н. Федоров. –Вологда : Инфра–Инженерия, 2015. –928 с. –Текст: непосредственный.

ПРИЛОЖЕНИЕ А

Описание интерфейсов функциональных блоков Library.fun

```
FUNCTION_BLOCK FB_Integrator
    VAR_INPUT
        in : REAL;
    END_VAR
    VAR_OUTPUT
        out : REAL;
    END_VAR
    VAR
        dt : REAL;
    END_VAR
END_FUNCTION_BLOCK

FUNCTION_BLOCK FB_Motor
    VAR_INPUT
        u : REAL;
    END_VAR
    VAR_OUTPUT
        w : REAL;
        phi : REAL;
    END_VAR
    VAR
        integrator : FB_Integrator;
        Tm : REAL;
        ke : REAL;
        dt : REAL;
    END_VAR
END_FUNCTION_BLOCK

FUNCTION_BLOCK FB_Regulator
    VAR_INPUT
        e : REAL;
    END_VAR
    VAR_OUTPUT
        u : REAL;
    END_VAR
    VAR
        k_p : REAL;
        k_i : REAL;
        integrator : FB_Integrator;
        iyOld : REAL;
        max_abs_value : REAL;
        dt : REAL;
    END_VAR
END_FUNCTION_BLOCK
```

ПРИЛОЖЕНИЕ Б

Переменные основной программы Variables.var

```
VAR
    fb_controller : FB_Regulator;
    fb_motor : FB_Motor;
    fb_motor_2 : FB_Motor;
    Speed : REAL;
    Enable : BOOL;
    counter : UDINT;
END_VAR
```

ПРИЛОЖЕНИЕ В

Инициализация основной программы Init.c

```
#include <bur/plctypes.h>

#ifdef _DEFAULT_INCLUDES
    #include <AsDefault.h>
#endif

void _INIT ProgramInit(void)
{
    fb_motor.ke=0.00096;
    fb_motor.Tm=0.04;
    fb_motor.dt=0.002;
    fb_motor.w=0.0;
    fb_motor.phi=0.0;
    fb_motor.integrator.out=0.0;

    fb_motor_2.ke=0.00096;
    fb_motor_2.Tm=0.04;
    fb_motor_2.dt=0.002;
    fb_motor_2.w=0.0;
    fb_motor_2.phi=0.0;
    fb_motor_2.integrator.out=0.0;

    fb_controller.k_p=0.0064;
    fb_controller.k_i=0.16;
    fb_controller.max_abs_value=24.0;
    fb_controller.dt=0.002;
    fb_controller.u=0.0;
    fb_controller.iyOld=0.0;
    fb_controller.integrator.out=0.0;

    Enable=0;
    Speed=0.0;
    counter=0;
}
```

ПРИЛОЖЕНИЕ Г

Основной цикл программы Cyclic.c

```
#include <bur/plctypes.h>

#ifdef _DEFAULT_INCLUDES
    #include <AsDefault.h>
#endif

void _CYCLIC ProgramCyclic(void)
{
    USINT i;

    if (!Enable) {
        counter=0;
        Speed=0.0;
        return;
    }

    counter++;

    /* Cyclic #3 = 50 ms: 400 cycles = 20 s. */
    if ((counter >= 400) && (counter < 800))
        Speed=60.0;
    else
        Speed=0.0;

    /* 50 ms / 2 ms = 25 model calculation steps. */
    for (i=0; i<25; i++) {
        fb_controller.e=Speed-fb_motor.w;
        FB_Regulator(&fb_controller);

        fb_motor.u=fb_controller.u;
        FB_Motor(&fb_motor);

        fb_motor_2.u=Speed * fb_motor_2.ke;
        FB_Motor(&fb_motor_2);
    }

    /* Full experiment: 60 seconds. */
    if (counter >= 1200)
        counter=0;
}
```

ПРИЛОЖЕНИЕ Д

Функциональный блок интегратора FB_Integrator.c

```
#include <bur/plctypes.h>
#include "Library.h"

void  FB_Integrator(struct  FB_Integrator*  inst)
{
    inst->out=inst->out  +  inst->in  *  inst->dt;
}
```

ПРИЛОЖЕНИЕ Е

Функциональный блок двигателя FB_Motor.c

```
#include <bur/plctypes.h>
#include "Library.h"

void FB_Motor(struct FB_Motor* inst)
{
    REAL e;

    e=inst->u/inst->ke-inst->w;  inst-
    >integrator.in=e/inst->Tm; inst-
    >integrator.dt=inst->dt;
    FB_Integrator(&inst->integrator);

    inst->w=inst->integrator.out;
    inst->phi=inst->phi +
        inst->w * 0.1047197551 * inst->dt;
}
```

ПРИЛОЖЕНИЕ Ж

Функциональный блок ПИ–регулятора FB_Regulator.c

```
#include <bur/plctypes.h>
#include "Library.h"

void FB_Regulator(struct FB_Regulator* inst)
{
    REAL p_term, sum_before_sat, u_sat;

    p_term=inst->e * inst->k_p;

    if (p_term > inst->max_abs_value)
        p_term=inst->max_abs_value;
    else if (p_term < -inst->max_abs_value)
        p_term=-inst->max_abs_value;

    inst->integrator.in=
        inst->e * inst->k_i + inst->iyOld;
    inst->integrator.dt=inst->dt;
    FB_Integrator(&inst->integrator);

    sum_before_sat=p_term + inst->integrator.out;

    if (sum_before_sat > inst->max_abs_value)
        u_sat=inst->max_abs_value;
    else if (sum_before_sat < -inst->max_abs_value)
        u_sat=-inst->max_abs_value;
    else
        u_sat=sum_before_sat;

    inst->iyOld=u_sat-sum_before_sat;
    inst->u=u_sat;
}
```